

Chương 6: Neural Networks

Báo cáo nhóm - Chương 6.1 đến 6.3

Bùi Huỳnh Tây - 24521589

Đoàn Hồng Bảo - 24520005

Nguyễn Thành Sơn - 24521532

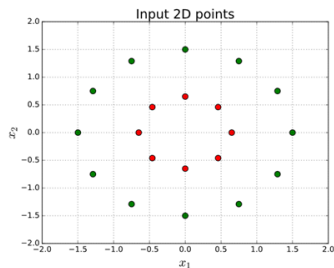
Nguyễn Quốc Phú - 24520025

Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM

Ngày 24 tháng 3 năm 2026

- 1 Introduction: Neural Networks
- 2 Feedforward Neural Networks
- 3 Representation

- Xét một bài toán phân loại trên mặt phẳng tọa độ 2D với tập dữ liệu "Vòng tròn đồng tâm" (Concentric Circles):
 - **Class 0:** Các điểm tụ tập ở lõi trung tâm.
 - **Class 1:** Các điểm phân bố thành vòng đai bao quanh bên ngoài.
- **Vấn đề:** Dữ liệu này **hoàn toàn phi tuyến tính** (Linearly Inseparable).
- **Bước ngoặt:** Để giải quyết vấn đề trên là lý do mà mạng Nơ-ron với các lớp ẩn và hàm kích hoạt phi tuyến ra đời!



Hình 1: Dữ liệu Vòng tròn đồng tâm phân tách phi tuyến tính.

Neural network hiện đại

1 Dữ liệu liên tục:

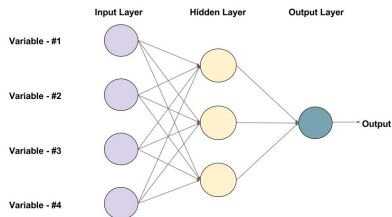
- Dùng véc-tơ x **Số thực**.

2 Hàm kích hoạt khả vi:

- Dùng hàm phi tuyến **khả vi** (Sigmoid, Tanh, ReLU).

3 Cơ chế tự học:

- **Lan truyền ngược (Backpropagation)**, cập nhật w, b .



An example of a Feed-forward Neural Network with one hidden layer (with 3 neurons)

Hình: một ví dụ minh họa về Neural Network.

- **Sự tương đồng:** Một đơn vị nơ-ron cơ bản sử dụng hàm Sigmoid về bản chất chính là một mô hình Hồi quy Logistic:

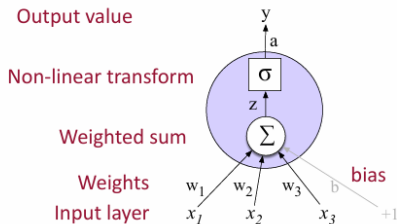
$$\hat{y} = \sigma(w \cdot x + b)$$

- **Sự vượt trội về lý thuyết:** Mạng nơ-ron là bộ phân loại mạnh mẽ hơn nhiều. Một mạng nơ-ron tối thiểu (chỉ cần một lớp ẩn) đã được chứng minh là có thể **xấp xỉ bất kỳ hàm số nào**.

- **Hồi quy Logistic:** Phụ thuộc vào việc xây dựng thủ công các **khuôn mẫu đặc trưng** phức tạp dựa trên kiến thức chuyên môn (domain knowledge).
- **Mạng nơ-ron:** Hướng tới việc bỏ qua các đặc trưng thủ công. Mô hình nhận **dữ liệu thô** và tự học cách trích xuất đặc trưng (*representation learning*) trong quá trình huấn luyện.
- -> Mạng nơ-ron đặc biệt hiệu quả và là công cụ đáng dẫn cho các tác vụ có **đủ lượng dữ liệu** để mô hình tự động học các đặc trưng này.

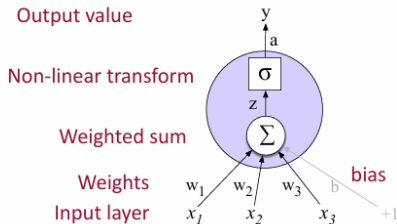
- **Input Layer (x_i):** Tiếp nhận dữ liệu đầu vào.
- **Weights (w_i):** Trọng số tương ứng cho mỗi đầu vào.
- **Bias (b):** Thành phần dịch chuyển độc lập.
- **Weighted Sum (z):** Tính tổng tất cả đầu vào đã nhân trọng số cộng với độ lệch:

$$z = \left(\sum_{i=1}^n w_i x_i \right) + b$$

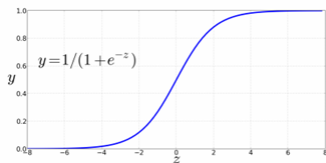


Ví dụ về 1 units của Neural network.

- Nếu chỉ dùng một hàm transform mạng nơ-ron nhiều lớp cũng chỉ tương đương với một lớp duy nhất.
- **Hạn chế:** Không thể học được các quy luật phức tạp, phi tuyến tính trong dữ liệu thực tế.
- **Giải pháp:** Cần một hàm tạo ra tính phi tuyến.



Ví dụ về 1 units của Neural network.



Sigmoid

- Thay thế Transform bằng hàm **Sigmoid** (σ):

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Đặc điểm:**

- Ánh xạ mọi giá trị về khoảng $(0, 1)$.
- Giúp mô hình có khả năng phân loại xác suất.
- Là bước đệm để tiến tới các hàm tối ưu hơn.

Các hàm kích hoạt hiện đại hơn

Dựa trên cấu trúc ở Hình 2, ta có thể linh hoạt thay thế σ bằng:

- **Tanh:**

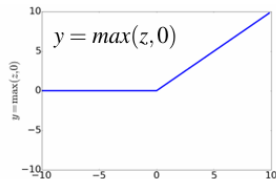
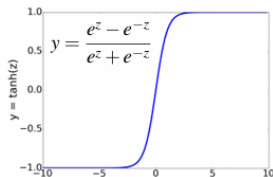
$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

⇒ Đưa dữ liệu về khoảng $(-1, 1)$, giúp chuẩn hóa dữ liệu quanh điểm 0 tốt hơn Sigmoid.

- **ReLU:**

$$f(z) = \max(0, z)$$

⇒ Giải quyết vấn đề **Triệt tiêu Gradient**, giúp huấn luyện các mạng cực sâu hiệu quả.



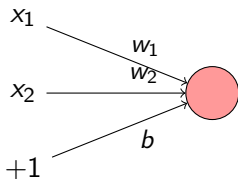
2 Feedforward Neural Networks

Perceptron

- **Perceptron:** Là đơn vị nơ-ron đơn giản nhất với đầu ra nhị phân (0 hoặc 1).
- **Cơ chế:** Sử dụng hàm bước (step function)

$$y = \begin{cases} 0 & \text{nếu } w \cdot x + b \leq 0 \\ 1 & \text{nếu } w \cdot x + b > 0 \end{cases} \quad (1)$$

- Một Perceptron đơn lẻ có thể mô phỏng hoàn hảo các cổng logic cơ bản như **AND** và **OR** bằng cách thay đổi trọng số W và bias b .



Mô hình Perceptron tổng quát

So sánh AND, OR và XOR

AND			OR			XOR		
x_1	x_2	y	x_1	x_2	y	x_1	x_2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

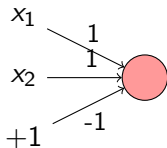
Bảng: Bảng chân trị của các hàm logic cơ bản

So sánh AND, OR và XOR

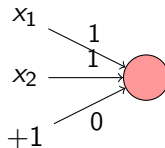
AND			OR			XOR		
x_1	x_2	y	x_1	x_2	y	x_1	x_2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

Bảng: Bảng chân trị của các hàm logic cơ bản

(a) Logical AND

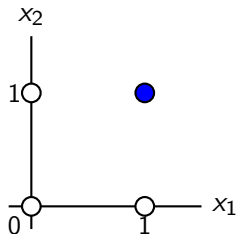


(b) Logical OR

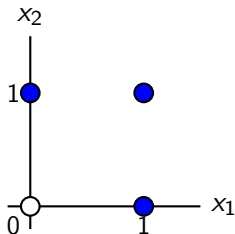


Ranh giới quyết định: AND, OR và XOR

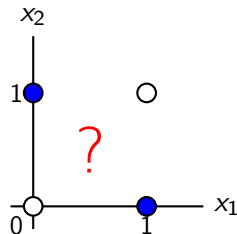
Một perceptron đơn lẻ tạo ra một ranh giới quyết định tuyến tính.



a) x_1 AND x_2



b) x_1 OR x_2

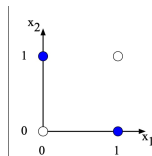
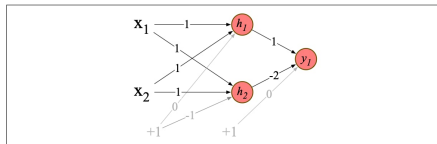


c) x_1 XOR x_2

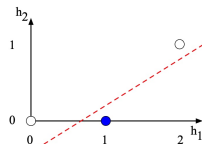
Kết luận từ ranh giới quyết định

Perceptron thất bại với bài toán **phi tuyến** (XOR) vì không thể dùng một đường thẳng duy nhất để phân tách đúng các điểm dữ liệu.

Solving XOR problems



a) The original x space



b) The new (linearly separable) h space

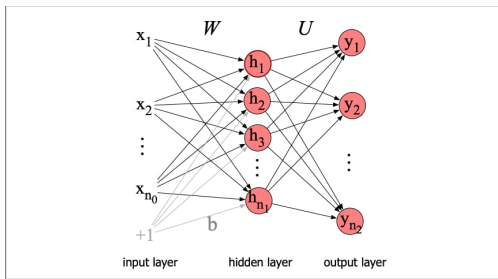
(a) Mạng Neural giải quyết bài toán XOR

(b) Biểu diễn không gian ẩn đã được tách

Lớp ẩn biến đổi dữ liệu sang không gian mới giúp XOR có thể phân tách tuyến tính.

Feedforward Neural Networks

- **Định nghĩa:** Mạng multi layers kết nối không chu trình, thông tin truyền thẳng một chiều từ layer thấp lên layer cao.
- **Cấu trúc:** Gồm 3 loại layer chính: Đầu vào (x), hidden layers (h) và đầu ra (y).
- **Đặc điểm:** Các layer kết nối đầy đủ (fully-connected)



- **Định nghĩa (Layer 0):** Là layer tiếp nhận dữ liệu thô đầu vào của mạng nơ-ron.
- **Cấu tạo:** Bao gồm các đơn vị đầu vào (input units), đại diện cho các đặc trưng (features) của quan sát.
- **Biểu diễn toán học:**
 - Thường được xử lý dưới dạng véc-tơ cột có kích thước $[n_0 \times 1]$.
- **Vai trò:** Cung cấp giá trị cơ sở để thực hiện các phép nhân ma trận trọng số ở các layer kế tiếp.

Hidden layers

- **Các thành phần và kích thước :**

- **Input (x):** Vector cột kích thước $[n_0 \times 1]$.
- **Weight matrix (W):** Ma trận trọng số kích thước $[n_1 \times n_0]$.
- **Bias (b):** Vector định hướng kích thước $[n_1 \times 1]$.

Công thức tính toán hidden layers (Layer 1)

Biểu diễn tại lớp ẩn h được tính bằng cách nhân ma trận trọng số với đầu vào, cộng bias và áp dụng hàm kích hoạt σ (thường là Sigmoid, ReLU hoặc Tanh):

$$h = \sigma(Wx + b) \quad (2)$$

- **Tính toán từng phần tử:** Giá trị của mỗi nút ẩn h_j được tính bằng:

$$h_j = \sigma \left(\sum_{i=1}^{n_0} W_{ji}x_i + b_j \right)$$

Output layers

- **output layers (Layer 2):** Sử dụng ma trận trọng số U [$n_2 \times n_1$] để tạo đầu ra trung gian z : $z = Uh$

Hàm Softmax và Xác suất (y)

Để chuyển z thành phân phối xác suất, ta dùng hàm Softmax:

$$y_i = \text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{n_2} \exp(z_j)} \quad (3)$$

Ví dụ thực tế về Softmax

Giả sử vector đầu ra trung gian z có các giá trị:

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

Sau khi áp dụng Softmax, ta thu được phân phối xác suất (tổng bằng 1):

$$y \approx [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

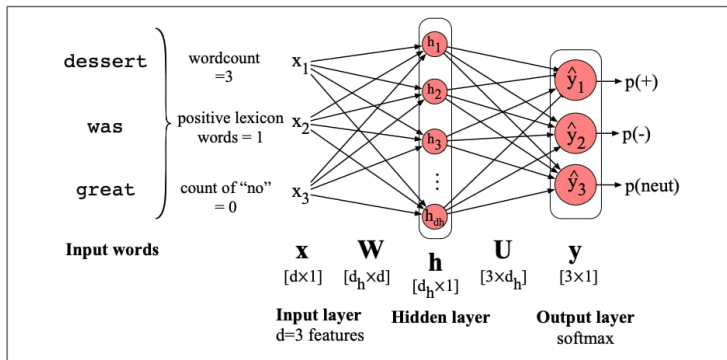
Feedforward Sentiment Classifier

$$x = [x_1, x_2, \dots, x_d]^T$$

$$h = \sigma(Wx + b)$$

$$z = Uh$$

$$\hat{y} = \text{softmax}(z)$$



3 Representation

Embeddings trong NLP

- Trước đây, các mô hình NLP thường dùng **đặc trưng thiết kế thủ công** (hand-built features), nhưng các mô hình neural hiện đại không còn phụ thuộc vào cách này.
- Thay vào đó, chúng **tự học đặc trưng từ dữ liệu** bằng cách biểu diễn từ (token) dưới dạng **embedding** (vector số).
- Có 2 loại embedding chính:
 - **Static embeddings** (Word2Vec, GloVe):
 - Mỗi từ có một vector cố định
 - Lưu trong một “từ điển” (embedding matrix)
 - **Contextual embeddings**:
 - Vector của từ thay đổi tùy theo ngữ cảnh
 - Được học trực tiếp trong quá trình huấn luyện (ví dụ: language modeling)

Ma trận Embedding (Embedding Matrix E)

- Ma trận embedding E đóng vai trò như một từ điển lưu trữ toàn bộ các static embeddings.
- Ma trận E có kích thước là $[|V| \times d]$.
- Trong đó:
 - $|V|$ là kích thước của toàn bộ từ vựng.
 - d là số chiều của từng vector embedding.
- Mỗi hàng trong ma trận E biểu diễn một token trong từ vựng dưới dạng một vector.

Trích xuất Embedding bằng One-hot Vector

- Một token có thể được biểu diễn bằng một **one-hot vector** có hình dạng $[1 \times |V|]$.
- Vector này chứa giá trị 1 tại vị trí chỉ số của từ đó trong từ vựng, và 0 ở tất cả các vị trí còn lại.
- Khi nhân one-hot vector với ma trận E , ta sẽ trích xuất được vector embedding (hàng tương ứng) của từ đó.

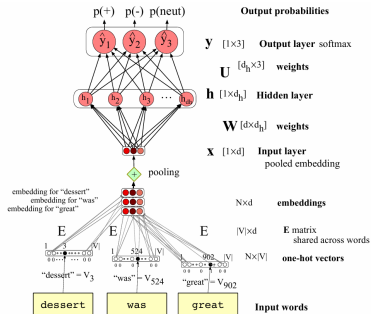
The diagram illustrates the extraction of a word embedding. On the left, a one-hot vector of size $1 \times |V|$ is shown, with a '1' at the third position (index 3) and '0's elsewhere. This is multiplied ($\times$) by an embedding matrix E of size $|V| \times d$, which has 3 rows highlighted in light green. The result is a single embedding vector of size $1 \times d$, also highlighted in light green.

- Cách làm này có thể mở rộng để biểu diễn một chuỗi gồm N token dưới dạng một ma trận các vector one-hot.

The diagram shows how a sequence of N tokens is represented. On the left, an $N \times |V|$ matrix of one-hot vectors is shown, with three rows highlighted in light green. This matrix is multiplied ($\times$) by the same embedding matrix E of size $|V| \times d$. The result is an $N \times d$ matrix of embeddings, with the corresponding three rows highlighted in light green.

Cách 1: Pooling (Gộp Vector)

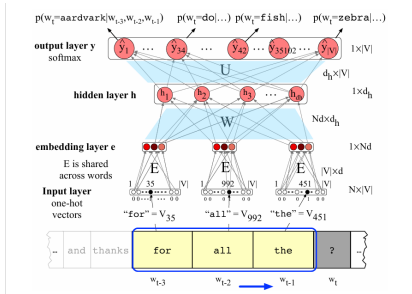
- **Ứng dụng:** Phân loại cảm xúc (Sentiment Analysis).
- **Nguyên lý:** Sự xuất hiện của từ quan trọng hơn thứ tự từ.
- **Cơ chế:** Gộp N vector embedding thành 1 vector duy nhất.
 - *Mean-pooling:* Lấy trung bình cộng.
 - *Max-pooling:* Lấy giá trị lớn nhất.
- **Ưu điểm:** Mạng nhỏ gọn, tính toán hiệu quả.



Hình: Cơ chế Pooling

Cách 2: Concatenation (Nối Vector)

- **Ứng dụng:** Mô hình ngôn ngữ (Language Modeling - dự đoán từ tiếp theo).
- **Nguyên lý:** Thứ tự và vị trí các từ trước đó là bắt buộc.
- **Cơ chế:** Nối tiếp N vector đầu vào (mỗi vector chiều d).
- **Kết quả:** Tạo ra một vector siêu dài kích thước $[1 \times dN]$ để giữ trọn vẹn thông tin ngữ cảnh.



Hình: Cơ chế Concatenation

Mô hình Ngôn ngữ Nơ-ron (Neural LM) vs. N-gram

- **Cơ chế hoạt động:** Neural LM dự đoán xác suất từ tiếp theo (w_t) dựa trên một cửa sổ ngữ cảnh gồm $N - 1$ từ trước đó:

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-N+1}, \dots, w_{t-1})$$

- **Sức mạnh tổng quát hóa (Generalization):** Sử dụng Embeddings thay vì định danh từ chính xác (word identity) giúp mô hình dự đoán tốt trên cả dữ liệu chưa từng gặp.
 - *Đã học:* "I have to make sure that the **cat** gets **fed**."
 - *Dự đoán:* "I forgot to make sure that the **dog** gets ..."
 - *Kết quả:* Neural LM đoán được "**fed**" vì vector của "cat" và "dog" tương đồng. N-gram sẽ thất bại nếu chưa từng thấy cụm "dog gets fed".
- **Hạn chế của Neural LM:**
 - Chậm hơn, cấu trúc phức tạp.
 - Tiêu tốn nhiều năng lượng và tài nguyên để huấn luyện.
 - Tính diễn giải kém (black-box) so với mô hình n-gram.
- → Với các tác vụ nhỏ và đơn giản, N-gram vẫn là một công cụ hiệu quả.

Quizz

Cảm ơn quý thầy cô và các bạn đã
lắng nghe!